
NPU simulator 진행 보고

Jul. 8, 2026

Park, Hyung chul

진행 보고 (1)

구현 여부	opcode	필요 연산	사용처	대체 방법
○	b0001_0100	Reduce Sum (행 합산)	RMSNorm (제곱합 → 평균), Softmax(지수합)	1 벡터와 내적
○	b0001_0101	Broadcast (스칼라→벡터)	RMSNorm(1/rms를 채널에 곱) Softmax(지수합으로 나눔)	1 벡터와 스칼라 곱으로 값 복제
○	다음 slide	Transpose	$Q \times K^T$	① 호스트에서 미리 전치 배치 ② 런타임은 원소 단위 복사
○	b0001_0110	부호 반전	RoPE(rotate half의 -x)	0벡터에서 뺄 (0-x)
○	b0001_0111	메모리 간 복사	RoPE(rotate half의 복사)	0벡터와 더함 (0+x)
○	b0001_1000	sin/cos	RoPE	사전 계산해 메모리에 배치 (혹은 LUT 이용)
○	b0100_0000 b0100_0001 b0100_0010	표준 활성화(SiLU/sigmoid)	SwiGLU FFN	현재 활성화가 $x^2 \cdot \text{sigmoid}(x)$ 형태로 부적합 → exp·덧셈·나눗셈·곱셈 조합으로 SiLU ($x \cdot \text{sigmoid}(x)$) 구성
✗	다음 slide	Reduce Max	Attention Softmax 최대값 빼기 (overflow 방지)	대체 방법 없음

transpose

- Transpose : K^T
 - 성균관대에서 요청하신 stride load 기능으로 구현됨
- 그래서, $Q * K^T$ 는
 - Load Q
 - Strided load K
 - Matrix multiplication
 - 로 구현될 수 있음.

Reduce max

- 서울과기대에서는 reduce max 기능은
 - matrix element들 중에서 최대값을 찾고,
 - 그 값을 matrix에 대해서 빼기 연산을 하는 것으로 이해함.
- 위의 이해대로라면
 - Vector min/max 명령어 (b0001_0010)을 이용해서
 - matrix의 각 row 또는 column에 대해서 각각 최대값을 구하고
 - 이 값들을 vector로 구현해서 최대값을 찾으면 됨.
 - 이렇게 제안하는 이유는
 - Min / max operation이 결국 모든 element를 비교하는 연산이므로 matrix 구현하는 것에 대한 장점이 없어서임.

진행 보고 (2)

구현 여부	opcode	필요 연산	사용처	현황·사유
✗	다음 slide	loop + 주소 auto-increment	GEMM	많은 타일 수로 인한 반복되는 명령어
○	b0100_0010	Matmul-accumulate ($C += A \cdot B$)	Tiled GEMM 부분합 누산	현재 matmul은 PE out을 덮어씀 → 타일마다 save-load-matrix_add 오버헤드 발생. ① 기본적으로 누산 형태로 진행 & accumulate buffer 초기화 지원 ② matmul에 누산 모드(더하기) 비트 추가
○	b1001_0000 b1001_1000	Strided load-store (행 간격 지정)	큰 행렬에서 64×64 타일 추출, Transpose 지원	현재 load는 연속(contiguous)만 지원 → 타일 추출에 per-row 로드 다수 또는 호스트 사전 재배치 필요

Loop + addr. Auto increment

- 서울과기대에서는 이 기능은
 - Matrix 전체에 대한 더하기, 빼기, 곱하기 등의 연산을 위해서
 - Matrix 중 일부들을 교체하면서 진행함에 있어서
 - Matrix 일부의 교체를 위한 주소에 대한 auto increment 라고 이해함.
- 위의 이해대로라면
 - 결국은 matrix 일부들을 계속 load 해야 하므로
 - Data load / save instruction 사용이 필요함.
 - 그러므로, data load / save instruction을 사용할 때 주소를 바꾼 후 사용하면 됨.

c-model 개선 요구 사항

구현 여부



1. 고정 용량 한계

- **한계** 데이터 버퍼 16KB(FP16 8,192개)·명령어 32K가 고정 크기
→ 8,192개 초과분은 무시되고, 초과 시 인접 프로그램 메모리를 침범해 실행이 손상됨
- **해결** 입력 파일 크기에 맞춰 버퍼·메모리를 동적 할당(또는 충분히 확장)



2. 프로그램 카운터 무한 증가

- **한계** 프로그램 카운터가 무한 증가 → 출력이 끝없이 발생해 결과 확인이 곤란함
- **해결** 종료 조건 추가 — 종료 OP 명령 도입, 로드된 명령어 길이 도달 시 종료



3. 파일 쓰기 기능 부재

- **한계** instruction별 출력만 확인 가능, 실행 후 데이터 버퍼 결과를 파일로 저장하는 기능이 없음
- **해결** 실행 종료 시 데이터 버퍼를 입력과 동일한 FP16 바이너리로 파일 출력
→ 단계 간 연결·결과 확인이 쉬워 컴파일러 구현에 유리

파일 쓰기

- Instruction 수행마다 데이터 버퍼를 모두 저장하면 파일 크기가 매우 커질 것으로 예상됨.
- Instruction 수행 후 데이터 버퍼의 내용은 load 명령어를 통해서 확인이 가능하므로 이 기능을 사용할 것을 권장함.

Instruction example codes

```
a.out
inst_0001_vector_add_immediate
inst_0002_vector_add_scalar
inst_0003_vector_add_vector
inst_0011_vector_sub_immediate
inst_0012_vector_sub_scalar
inst_0013_vector_sub_vector
inst_0021_vector_logical_AND_immediate
inst_0022_vector_logical_OR_scalar
inst_0023_vector_logical_NOT
inst_0024_vector_logical_XOR_immediate
inst_0025_vector_logical_NAND_scalar
inst_0026_vector_logical_NOR_vector
inst_0031_vector_shift_immediate
inst_0032_vector_shift_scalar
inst_0033_vector_shift_vector
inst_0041_vector_multiply_immediate
inst_0042_vector_multiply_scalar
inst_0043_vector_multiply_vector
inst_0051_vector_divide_immediate
inst_0052_vector_divide_scalar
inst_0053_vector_divide_vector
inst_0061_vector_mul_add_immediate
inst_0062_vector_mul_add_scalar
inst_0063_vector_mul_add_vector
inst_0071_vector_move_immediate
inst_0072_vector_move_scalar
inst_0073_vector_move_vector
inst_0081_vector_square-root
inst_0091_vector_compare_immediate
inst_0092_vector_compare_scalar
inst_0093_vector_compare_vector
inst_0102_vector_min_scalar
inst_0103_vector_min_vector
inst_0112_vector_max_scalar
inst_0113_vector_max_vector
inst_0123_vector_exponential
inst_0124_vector_type_convert_0
inst_0125_vector_type_convert_1
inst_0126_vector_reduce_sum
inst_0127_vector_broadcast_immediate
inst_0128_vector_broadcast_scalar
inst_0129_vector_sign_inversion
inst_0130_vector_copy
inst_0131_vector_cos
inst_0132_vector_sin
inst_1001_matrix_add_immediate
inst_1002_matrix_add_scalar
inst_1003_matrix_add_matrix
inst_1004_matrix_add_matrix_w_act
inst_1011_matrix_sub_immediate
inst_1012_matrix_sub_scalar
inst_1013_matrix_sub_matrix
inst_1014_matrix_sub_matrix_w_act
inst_1021b_matrix_mul_immediate_w_act
inst_1021_matrix_mul_immediate
inst_1022b_matrix_mul_scalar_w_act
inst_1022_matrix_mul_scalar
inst_1023b_matrix_mul_matrix_w_act
inst_1023_matrix_mul_matrix
inst_1031_matrix_move_immediate
inst_1032_matrix_move_scalar
inst_1033_matrix_move_matrix
inst_1034_matrix_stride_load_store
program_memory.bin
w_support_program_mem_gen.cpp
```

- 포함된 .tar 파일에서 b_program directory에서 위 노란 부분들이
- 이번에 추가 또는 수정된 명령어에 대한 example code가 있는 directory 임.